Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №18

ТЕМА: (Нова) Покрокове відлагодження коду (Debugging) та трасування виконання циклічних алгоритмів.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з поняттям налагодження (debugging) програми та типами помилок;
- вивчити прийом трасування — покрокового аналізу зміни значень змінних під час виконання циклу;
- навчитися складати трасувальну таблицю для циклічного алгоритму;
- розвинути навички самостійного пошуку логічних помилок у програмі.

Теоретичні відомості

1. Типи помилок у програмах

- Синтаксичні помилки (syntax errors) — порушення правил запису мови (пропущена двокрапка, дужка, неправильний відступ). Інтерпретатор виявляє їх одразу і не дає програмі запуститися.
- Помилки виконання (runtime errors) — виникають під час роботи програми (наприклад, ділення на нуль, звернення до неіснуючого елемента). Програма аварійно завершується повідомленням про помилку.
- Логічні помилки (logic errors) — найпідступніший тип: програма працює без аварійного завершення, але видає невірний результат через помилку в самому алгоритмі (наприклад, неправильна умова циклу чи переплутаний порядок дій).

Саме логічні помилки найважче знайти, оскільки програма не повідомляє про них сама — потрібно самостійно проаналізувати, чому результат виявився неправильним. Основний інструмент для цього — трасування.

2. Трасування (tracing) циклу

Трасування — це покроковий запис значень усіх важливих змінних програми після виконання кожної команди чи кожної ітерації циклу. Трасувальну таблицю можна скласти на папері (для перевірки алгоритму «вручну», без комп'ютера) або спостерігати значення змінних безпосередньо в середовищі програмування за допомогою дебагера (налагоджувача).

Розглянемо приклад — програму, яка обчислює суму чисел від 1 до 4:

```
total = 0
i = 1
while i <= 4:
    total = total + i
    i = i + 1
print(total)
```

Трасувальна таблиця для цієї програми виглядає так:

Ітерація	**Умова i <= 4**	**total (до)**	**total (після)**	**i (після)**
до циклу	—	—	0	1
1	True	0	1	2
2	True	1	3	3
3	True	3	6	4
4	True	6	10	5
5	False (цикл завершено)	—	—	—

Складання такої таблиці дозволяє перевірити правильність алгоритму ще до запуску програми на комп'ютері, а також знайти момент, у якому значення змінної відхиляється від очікуваного.

3. Використання print() для налагодження

Найпростіший спосіб побачити, що відбувається всередині циклу під час реального виконання програми, — тимчасово додати команди print() із проміжними значеннями змінних усередині тіла циклу:

```
total = 0
i = 1
while i <= 4:
    total = total + i
    print(f'Крок {i}: total = {total}')  # налагоджувальне виведення
    i = i + 1
print('Підсумок:', total)
```

Такі тимчасові налагоджувальні print() варто прибирати (або коментувати) після того, як помилку знайдено й виправлено, щоб не захаращувати остаточний вивід програми.

4. Покроковий дебагер у середовищі програмування

Більшість сучасних середовищ програмування (PyCharm, VS Code, Thonny) мають вбудований дебагер, що дозволяє: поставити точку зупинки (breakpoint) на потрібному рядку коду; запустити програму в режимі налагодження; виконувати програму покроково (step over / step into); спостерігати поточні значення всіх змінних у спеціальній панелі, не додаючи жодного print() у сам код.

Хід роботи

Варіант і вимоги до здачі

а)N — ваш порядковий номер у списку групи (журнал). Дані завдань залежать від N:

Завдання	**Дані варіанта**	**Приклад N = 7**
1	факторіал для n = 4 + N mod 5	n = 6 (720)
3	програма з помилкою: А — якщо N непарне, Б — якщо N парне; n = 3 + N mod 4	програма А, n = 6
5	цикл для чисел від 1 до m, m = 8 + N mod 4	m = 11

б)Програми здавайте окремими файлами Прізвище_ПР18_зN.py, код — текстом, а не фото. Файли з прикладів із теорії здавати не потрібно.

в)Усі трасувальні таблиці та скріншоти оформлюйте в одному звіті (документ Word). Таблицю в зошиті чи на фото приймаємо тільки як додаток до таблиці у звіті. Для завдання 2 у звіті обов'язковий скріншот вікна дебагера з точкою зупинки й панеллю значень змінних; без нього завдання 2 не зараховується.

г)Запускайте програми й дебагер на своєму комп'ютері (IDLE, PyCharm, Thonny тощо), а не в онлайн-компіляторі.

д)У коді залиште короткі коментарі власними словами перед основними блоками.

е)Будьте готові пояснити будь-який рядок свого коду й свою трасувальну таблицю та внести невелику зміну (див. розділ «Захист роботи»).

1.Створіть файл Прізвище_ПР18_з1.py із програмою обчислення факторіалу числа n (за допомогою циклу while; n — за вашим варіантом). Складіть у звіті трасувальну таблицю (стовпці: крок, i, result, умова i ≤ n), аналогічну наведеній у теоретичній частині. Результат програми має збігатися з останнім рядком таблиці.

Приклад (N = 7): n = 6 → 720.

2.У середовищі програмування поставте точку зупинки (breakpoint) на рядку з тілом циклу та запустіть програму з завдання 1 у режимі налагодження. Покроково прослідкуйте за зміною значень змінних і порівняйте зі своєю трасувальною таблицею. Додайте до звіту скріншот вікна дебагера з точкою зупинки та значеннями змінних на одному з кроків.

3.Розгляньте програму з навмисною логічною помилкою за вашим варіантом (А або Б, див. нижче). Складіть для неї трасувальну таблицю для n за вашим варіантом, знайдіть помилку, поясніть її в коментарі та здайте виправлену версію програми (файл Прізвище_ПР18_з3.py).

Програма А: обчислення факторіалу n.

```
n = int(input('n = '))
result = 1
i = 1
while i < n:
    result = result * i
    i = i + 1
print(result)
```

Програма Б: сума чисел від 1 до n.

```
n = int(input('n = '))
total = 0
i = 1
while i <= n:
    i = i + 1
    total = total + i
print(total)
```

Приклад (N = 7): програма А, n = 6 виводить 120 замість 720. Для програми Б при n = 4 виводиться 14 замість 10.

4.Додайте до власної програми з попередніх занять (наприклад, обчислення суми ряду) тимчасові налагоджувальні print() усередині циклу, переконайтеся, що проміжні значення відповідають очікуваним, після чого приберіть налагоджувальні виведення. До звіту додайте скріншот виведення з налагоджувальними print(), а в файлі залиште остаточну версію без них.

5.Складіть трасувальну таблицю для циклу з командою continue (виведення лише непарних чисел від 1 до m, m — за вашим варіантом; стовпці: i, умова i % 2 == 0, виконано continue (так/ні), що виведено) і порівняйте кількість фактичних ітерацій циклу з кількістю виведених значень. Напишіть програму й здайте її файлом Прізвище_ПР18_з5.py.

Приклад (N = 7): m = 11 — 11 ітерацій, виведено 6 значень (1, 3, 5, 7, 9, 11).

6. Завдання підвищеної складності*. Складіть трасувальну таблицю для вкладеного циклу (побудова таблиці множення 3×3), відобразивши в таблиці значення обох лічильників (i та j) на кожному кроці внутрішнього циклу (стовпці: крок, i, j, i·j). Таблиця має містити 9 рядків. Напишіть програму й здайте її файлом Прізвище_ПР18_з6.py.

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови для даних вашого варіанта — 100% балів; програма працює, але є суттєве зауваження (наприклад, немає трасувальної таблиці у звіті чи скріншота дебагера, не виправлено помилку) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити значення n чи пояснити крок трасувальної таблиці). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Назвіть три типи помилок у програмах та їхні відмінності.

2. Чому логічні помилки вважаються найскладнішими для виявлення?

3. Що таке трасування і навіщо воно потрібне?

4. Які стовпці зазвичай містить трасувальна таблиця для циклу?

5. Які переваги дає використання вбудованого дебагера порівняно з тимчасовими print()?